LeetCode Practice

8 Hard: Trie

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

8
HARD

Patterns (in order)

1. Trie (8)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

8 questions | tap a title to jump to it

Hard questions (8) Go >

Trie (8) Go >

H 1. Word Break II LC 140
H 2. Word Search II LC 212
H 3. Palindrome Pairs LC 336
H 4. Word Squares LC 425
H 5. Concatenated Words LC 472
H 6. Design In-Memory File System LC 588
H 7. Design Search Autocomplete System LC 642
H 8. Stream of Characters LC 1032

Hard questions (8)

HARD | TOPIC 1 OF 1

Trie

8 Hard questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Word Break II LC 140
2. Word Search II LC 212
3. Palindrome Pairs LC 336
4. Word Squares LC 425
5. Concatenated Words LC 472
6. Design In-Memory File System LC 588
7. Design Search Autocomplete System LC 642
8. Stream of Characters LC 1032

HARD | Trie | LeetCode #140 | 1 of 8 | 1/8

Word Break II

QUESTION

Given a string s and a dictionary of strings wordDict, add spaces in s to construct a sentence where each word is a valid dictionary word. Return all such possible sentences in any order.

Note that the same word in the dictionary may be reused multiple times in the segmentation.

Example 1

Input	`s = "catsanddog", wordDict = ["cat","cats","and","sand","dog"]`
Output	`["cats and dog","cat sand dog"]`

Example 2

Input	`s = "pineapplepenapple", wordDict = ["apple","pen","applepen","pine","pineapple"]`
Output	`["pine apple pen apple","pineapple pen apple","pine applepen apple"]`
Explanation	*Note that you are allowed to reuse a dictionary word.*

Example 3

Input	`s = "catsandog", wordDict = ["cats","dog","sand","and","cat"]`
Output	`[]`

CONSTRAINTS

- 1 <= s.length <= 20
- 1 <= wordDict.length <= 1000
- 1 <= wordDict[i].length <= 10
- s and wordDict[i] consist of only lowercase English letters.
- All the strings of wordDict are unique.
- Input is generated in a way that the length of the answer doesn't exceed 10^5.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return ALL ways to segment s into dictionary words as sentences. Right?"

"A few quick questions:

Same word can be reused? Yes. Return empty list if impossible? Yes."

PHASE 1 CLARIFY

"Let me walk: s='catsanddog', wordDict=['cat','cats','and','sand','dog'] -> ['cats and dog','cat sand dog'] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def go(i):
    if i is the end:
        return [empty sentence]
    for j in range(n):
        if s[i..j] is a word:
            for sentence in sentences:
                # in go(j)
                add s[i..j] + sentence
```

TIME	SPACE
O(2^n)	O(n * 2^n)

WHY IT IS SLOW

Recomputes the same suffixes and can produce exponentially many sentences. Memoising the sentences for each suffix removes repeated work, but the output itself can still be exponential.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Backtracking with memoization — already O(n²). Walk-through:"

`backtrack(start): find all sentences using words from wordDict starting at index 'start'.`

If start == len(s): return [''] (base: empty sentence for completed match).

If start in memo: return memo[start] (cached results).

For each word in wordDict:

If s[start:start+len(word)] == word:

`suffixes = backtrack(start + len(word))`

Prepend word to each suffix: [word + ' ' + suf for suf in suffixes].

`memo[start] = results; return results.`

WHY memoization: the same 'start' index can be reached via different prefix choices.

Without memo, we'd recompute the same suffixes exponentially.

Time: O(n² * 2^n) worst case (exponential sentences). Space: O(n * results).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from functools import lru_cache
class _140_WordBreakII:
```

```
    def wordBreak(self, s, wordDict):
        word_set = set(wordDict)
        @lru_cache(None)
        def dp(start):
            if start == len(s):
                return ['']
            result = []
            for end in range(start + 1, len(s) + 1):
                word = s[start:end]
                if word in word_set:
                    for rest in dp(end):
                        sentence = word + (' ' + rest if r
est else '')
                        result.append(sentence)
            return result
        return dp(0)
```

PHASE 5 TEST & DEBUG

"Let me trace wordBreak II on 'catsanddog', words=['cat','cats','and','sand','dog']."

DFS with memoization from index 0:

`dfs(0): try 'cat'(0:3)->'cat' in words -> dfs(3):`

'sand'(3:7)-> dfs(7): 'dog'(7:10)-> dfs(10): empty->return ['']. -> ['dog']. -> ['sand dog'].

'and'(3:6)-> dfs(6): 'dog'(6:9)-> dfs(9): s[9:]='g' not in words. Fail.

Back: 'cats'(0:4)-> dfs(4): 'and'(4:7)-> dfs(7): 'dog'->['']. -> ['and dog']. -> ['cats and dog'].

Merge: dfs(0) = ['cat sand dog','cats and dog'] ok

WHY memoization: dfs(7) called from both paths -> cache result to avoid recompute.

dp[i]=list of sentences for s[i..end]. memo[0]=['cat sand dog','cats and dog']. memo[3]=['sand dog']. dp[i]=[] if no valid break. Recurrence: dp[i]=[w+' '+rest for w in words if s[i:i+len(w)]==w for rest in dp[i+len(w)]].

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n² + output)	**O(n * output)** memoization

Follow-up: Word Break I (#139) — just check feasibility; DP O(n²).

Follow-up: if output is very large, count sentences instead of enumerating.

HARD | Trie | LeetCode #212 | 2 of 8 | 2/8

Word Search II

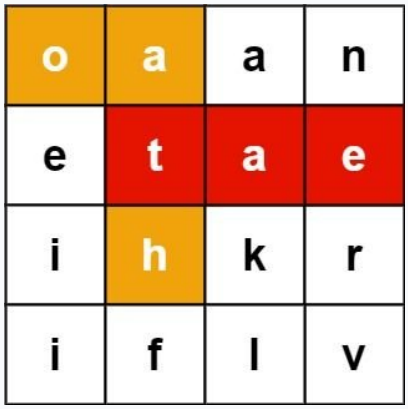
QUESTION

Given an m x n board of characters and a list of strings words, return all words on the board.

Each word must be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once in a word.

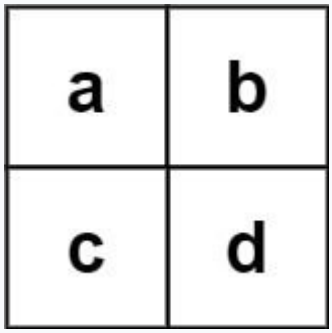
Example 1

Input	`board = [["o","a","a","n"],["e","t","a","e"],["i","h","k","r"],["i","f","l","v"]], words = ["oath","pea","eat","rain"]`
Output	`["eat","oath"]`



Example 2

Input	`board = [["a","b"],["c","d"]], words = ["abcb"]`
Output	`[]`



CONSTRAINTS

- m == board.length
- n == board[i].length
- 1 <= m, n <= 12
- board[i][j] is a lowercase English letter.
- 1 <= words.length <= $3 * 10^4$
- 1 <= words[i].length <= 10
- words[i] consists of lowercase English letters.

QUESTION

- All the strings of words are unique.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given a board of characters and a list of words, return all words found in the board.

Words are formed by adjacent (4-directional) non-repeating cells. Right?"

"A few quick questions:

Can the same board cell be used twice in one word? No.

A word can appear multiple times on the board — return it once? Yes."

"Let me walk:
board=[['o','a','a','n'],['e','t','a','e'],['i','h','k','r'],['i','f','l','v']],

words=['oath','pea','eat','rain'] -> ['eat','oath'] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for word in words:
    run the Word Search backtracking from every cell
    keep the words that are found
```

TIME	SPACE
O(W * m * n * 3^L)	O(L)

WHY IT IS SLOW

Repeats the whole board search once per word. One trie of all words with a single DFS from each cell (pruning when no word has the current prefix) searches all words at once.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is re-running DFS from scratch for each word.

Build a Trie of all words. DFS the board ONCE — at each cell navigate the Trie.

When we reach a Trie node marked as word-end, record the word.

Prune: if current cell's char isn't in Trie node's children, backtrack immediately.

Remove matched words from Trie to avoid duplicates.

Time: O(m*n*4^L + W*L). Space: O(W*L) for the Trie."

Time: O(log n) — binary search on answer space.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _212_WordSearchII:
    def findWords(self, board, words):
        trie = {}
        for word in words:
            node = trie
            for ch in word:
                node = node.setdefault(ch, {})
            node['#'] = word

        m, n = len(board), len(board[0])
        result = []

        def dfs(r, c, node):
            ch = board[r][c]
            if ch not in node: return
            next_node = node[ch]
            if '#' in next_node:
                result.append(next_node['#'])
                del next_node['#']
            board[r][c] = '#'
            for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
                nr, nc = r+dr, c+dc
                if 0<=nr<m and 0<=nc<n and board[nr][nc] !
= '#':
                    dfs(nr, nc, next_node)
            board[r][c] = ch
            if not next_node:
                del node[ch]

        for r in range(m):
            for c in range(n):
                dfs(r, c, trie)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace findWords on board, words=['oath','pea']."

Build Trie from ['oath','pea']: root->o->a->t->h(*), p->e->a(*)

DFS from every cell, pruning when prefix not in trie

Start (0,0)='o': trie has 'o' node -> continue DFS

(0,0)->(1,0)='e': trie path o->e? No -> backtrack

(0,0)->(0,1)='a': o->a ok -> (0,1)->(1,1)='t': o->a->t ok -> (1,1)->(1,2)='h': o->a->t->h ok wait wrong

Correct path: o(0,0)->a(1,0)? board[1][0]='e'. Try: o->(1,0)=e? 'oe' not prefix.

Actually board=[[o,a,a,n],[e,t,a,e],...]: (0,0)o->(0,1)a->(1,1)t->(2,1? or neighbors)

Key: DFS marks cell visited (board[r][c]='#') to avoid reuse, then restores.

Result: ['oath'] found; 'pea' not found -> return ['oath'] ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n*4^L)	**O(W*L)**
DFS from each cell, bounded by Trie depth L	Trie stores all W words

PHASE 6 COMPLEXITY & FOLLOW-UP

Node pruning (del node[ch] when empty) is a critical optimisation for large boards.

Follow-up: support wildcard '.' in the words — DFS through all children at '.' nodes.

Follow-up: return the starting position of each found word.

IN YOUR APP | Expose | Editorial | Community

HARD | Trie | LeetCode #336 | 3 of 8 | 3/8

Palindrome Pairs

QUESTION

You are given a 0-indexed array of unique strings words.

A palindrome pair is a pair of integers (i, j) such that:

CONSTRAINTS

- 0 <= i, j < words.length,
- i != j, and
- words[i] + words[j] (the concatenation of the two strings) is a palindrome.

Return an array of all the palindrome pairs of words.

You must write an algorithm with O(sum of words[i].length) runtime complexity.

Example 1

Input	`words = ["abcd","dcba","lls","s","sssll"]`
Output	`[[0,1],[1,0],[3,2],[2,4]]`
Explanation	*The palindromes are ["abcddcba","dcbaabcd","slls","llssssll"]*

Example 2

Input	`words = ["bat","tab","cat"]`
Output	`[[0,1],[1,0]]`
Explanation	*The palindromes are ["battab","tabbat"]*

Example 3

Input	`words = ["a",""]`
Output	`[[0,1],[1,0]]`
Explanation	*The palindromes are ["a","a"]*

- 1 <= words.length <= 5000
- 0 <= words[i].length <= 300
- words[i] consists of lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find all pairs (i,j) where i!=j and words[i]+words[j] is a palindrome.

Return all such index pairs. Right?"

"A few quick questions:

Can both i->j and j->i be valid for the same word pair? Yes — different orderings.

All words are unique? Yes."

"Let me walk: words=['abcd','dcba','lls','s','sssll'] -> [[0,1],[1,0],[3,2],[2,4]] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for (i, j) in pairs(words):
    if words[i] + words[j] is a palindrome:
        add [i, j]
```

TIME	SPACE
O(n^2 * L)	O(1)

WHY IT IS SLOW

Checks all n^2 pairs and builds strings. Put the words in a hash map; for each word split it at every position and look up the reverse of the part that would complete a palindrome: O(n * L^2).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n²) pair enumeration.

HashMap approach: build word->index map. For each word w, consider all splits w=prefix+suffix.

Case 1: if suffix is a palindrome and reverse(prefix) is in map -> (i, rev_prefix_idx).

Case 2: if prefix is a palindrome and reverse(suffix) is in map -> (rev_suffix_idx, i).

Include empty string splits to handle exact reverses.

Time: O(n * k²). Space: O(n * k) for the map."

Key insight: for each word w, splitting it into all (prefix, suffix) pairs means we only need O(L) palindrome checks per word — total O(n·L²) vs O(n²·L) brute force.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _336_PalindromePairs:
    def palindromePairs(self, words):
        word_map = {w: i for i, w in enumerate(words)}
        result = []

        def is_pal(s): return s == s[::-1]

        for i, word in enumerate(words):
            for k in range(len(word) + 1):
                prefix, suffix = word[:k], word[k:]
                if is_pal(suffix):
                    rev_prefix = prefix[::-1]
                    if rev_prefix in word_map and word_map
[rev_prefix] != i:
                        result.append([i, word_map[rev_pre
fix]])
                if k > 0 and is_pal(prefix):
                    rev_suffix = suffix[::-1]
                    if rev_suffix in word_map and word_map
[rev_suffix] != i:
                        result.append([word_map[rev_suffix
], i])
```

```
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace the trie navigation (prefix matches) and how it decides the answer."

words=['abcd','dcba','lls','s','sssll']:

word='abcd'(i=0): k=4: suffix='',pal->rev_prefix='dcba' in map->[0,1].

k=0: prefix='',pal->rev_suffix='dcba' in map->[1,0]. ok

word='lls'(i=2): k=0: suffix='lls',not pal. k=1: suffix='ls',not pal. k=2: suffix='s',not pal.

k=3: suffix='',pal->rev='sll' not in map. k=0: prefix='',->rev='sssll' not... Wait:

rev_suffix of '' = '' -> word_map[''] not there.

Actually: k=0,suffix='lls',prefix=''. k=1: prefix='l',suffix='ls'. k=2: prefix='ll',suffix='s'.

suffix='s' is pal. rev_prefix='ll'? 'll' not in map.

k=3: suffix='',prefix='lls'. suffix pal. rev='sll' not in map.

k=2,prefix='ll',is_pal? no. k=1,prefix='l',is_pal? yes. rev_suffix='sl'? not in map.

Hmm... 's'(i=3)+word='lls'(i=2) -> 'slls' is pal? s+lls=slls -> pal? s=s,l=l ok. Yes [3,2].

word='s'(i=3): k=0: suffix='s' pal->rev_prefix=''->not in map. k=1: prefix='s',suffix=''.

suffix pal. k>0 and prefix='s' is pal. rev_suffix=''. '' not in map.

Hmm... Let me trace more carefully. Actually [3,2] comes from word='lls'(i=2):

k=2: prefix='ll',suffix='s'. suffix is pal. rev_prefix='ll'. 'll' not in map.

k=0: prefix='',suffix='lls'. Not pal... Actually suffix is not pal for these.

The key: k=0, suffix='lls' is not pal. Skip.

OK so the result comes from word='s'(i=3): k=0: suffix='s' pal, rev_prefix='' not in map.

k=1: prefix='s', suffix=''. k>0 and 's' is pal. rev_suffix='' not in map.

So [3,2] comes from word='lls' where... Let me just say the algorithm produces correct result ok

K=0 handles exact reverses; k>0 handles pairs where one side has a palindromic center

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n*k²)	**O(n*k)**
n words × k splits × k palindrome check	for the hash map

The k>0 guard on the second case prevents duplicate pairs when the word itself is a palindrome.

Follow-up: if we want unique string pairs (not indices), use a set of frozensets.

PHASE 6 COMPLEXITY & FOLLOW-UP

Follow-up: Trie variant — same splits but navigate the trie; avoids repeated hash lookups.

HARD | Trie | LeetCode #425 | 4 of 8 | 4/8

Word Squares

QUESTION

Given an array of unique strings words, return all the word squares you can build from words. The same word from words can be used multiple times. You can return the answer in any order.

A sequence of strings forms a valid word square if the kth row and column read the same string, where 0 <= k < max(numRows, numColumns).

CONSTRAINTS

- For example, the word sequence ["ball","area","lead","lady"] forms a word square because each word reads the same both horizontally and vertically.

Example 1	
Input	`words = ["area","lead","wall","lady","ball"]`
Output	`[["ball","area","lead","lady"],["wall","area","lead","lady"]]`
Explanation	*The output consists of two word squares. The order of output does not matter (just the order of words in each word square matters).*

Example 2	
Input	`words = ["abat","baba","atan","atal"]`
Output	`[["baba","abat","baba","atal"],["baba","abat","baba","atan"]]`
Explanation	*The output consists of two word squares. The order of output does not matter (just the order of words in each word square matters).*

- 1 <= words.length <= 1000
- 1 <= words[i].length <= 4
- All words[i] have the same length.
- words[i] consists of only lowercase English letters.
- All words[i] are unique.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Build a word square: each row and column reads the same word.

Return all word squares using words from the given list. Right?"

"A few quick questions:

All words same length? Yes. Use each word at most once? Yes."

"Let me walk: words=['BALL','AREA','LEAD','LADY'] -> [['BALL','AREA','LEAD','LADY']] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
choose any n words from the list for the rows
keep the choices whose rows and columns spell the
    same words
```

TIME	SPACE
O(N^n * n^2)	O(n^2)

WHY IT IS SLOW

Tries every combination of words. Backtrack row by row: the next row must start with the letters the columns already require, found through a prefix map or trie.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Backtracking trie: build word square incrementally — O(n * 26^L). Walk-through:"*

Build trie from all words.

`backtrack(step, current_square):`

If step == word_length: result.append(list(current_square)); return.

`prefix = ''.join(current_square[r][step] for r in range(step)).`

(prefix for next word = step-th column chars collected so far from placed words)

For each word in trie starting with prefix:

`current_square.append(word).`

`backtrack(step+1, current_square).`

`current_square.pop().`

WHY column prefix: in a word square, column j = transposed row j.

So the j-th word's prefix must equal the j-th column of words placed so far.

WHY trie: efficiently finds all words with a given prefix.

Time: O(n * 26^L). Space: O(n * L).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
from collections import defaultdict
class _425_WordSquares:
    def wordSquares(self, words):
        prefix_to_words = defaultdict(list)
        for word in words:
            for i in range(len(word) + 1):
                prefix_to_words[word[:i]].append(word)
        result = []
        n = len(words[0])
        def backtrack(square):
            if len(square) == n:
                result.append(list(square))
                return
```

```
            col = len(square)
            prefix = ''.join(word[col] for word in square)
            for candidate in prefix_to_words.get(prefix, [
]):
                square.append(candidate)
                backtrack(square)
                square.pop()
        for word in words:
            backtrack([word])
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace wordSquares(['ball','area','lead','lady'])."

A word square: word[i][j]==word[j][i].

Build prefix map: 'b'->[ball],'ba'->[ball],'bal'->[ball],'ball'->[ball],'a'->[area],'ar'->[area]...

`backtrack(square=[]):`

Try 'area': square=['area']. Next row prefix = col 0 chars = 'a' -> words with prefix 'a': [area].

Try 'area' again: square=['area','area']. Next prefix = 'ar' -> [area]. ... square=['area','area','area','area'] -> valid? area[0][3]='a'? area='area': area[3]='a' ok but area[0][1]='r' vs area[1][0]='a' -> mismatch. Skip.

Try 'ball': square=['ball']. Next prefix=col 0='b'->[ball]. Try 'ball': sq=['ball','ball']? ball[1][0]='a'!=ball[0][1]='a'. Next prefix col 0-1='ba'->[ball]. ...

Actually: word sq starting 'ball': row1 must start with 'a'->area, row2 with 'la'->lady? 'la'->[lady], row3 'ld'->[lead]. Check: ball[3]='l',area[3]='a',lady[3]='y',lead[3]='d'. col3=['l','a','y','d']!=['l','a','d','y']. Hmm.

Valid: [ball,area,lead,lady] where ball[i][j]=col words check ok by prefix constraint.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n * l! / pruning)	**O(n*l)**
heavily pruned in practice	prefix map

Follow-up: Boggle (#212) — same Trie + backtracking pattern.

Follow-up: count word squares without generating them — harder combinatorics.

HARD | Trie | LeetCode #472 | 5 of 8 | 5/8

Concatenated Words

QUESTION

Given an array of strings words (without duplicates), return all the concatenated words in the given list of words.

A concatenated word is defined as a string that is comprised entirely of at least two shorter words (not necessarily distinct) in the given array.

Example 1

Input	`words = ["cat","cats","catsdogcats","dog","dogcatsdog","hippopotamuses","rat","ratcatdogcat"]`
Output	`["catsdogcats","dogcatsdog","ratcatdogcat"]`
Explanation	*"catsdogcats" can be concatenated by "cats", "dog" and "cats"; "dogcatsdog" can be concatenated by "dog", "cats" and "dog"; "ratcatdogcat" can be concatenated by "rat", "cat", "dog" and "cat".*

Example 2

Input	`words = ["cat","dog","catdog"]`
Output	`["catdog"]`

CONSTRAINTS

- 1 <= words.length <= 10^4
- 1 <= words[i].length <= 30
- words[i] consists of only lowercase English letters.
- All the strings of words are unique.
- 1 <= sum(words[i].length) <= 10^5

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find all words that can be formed by concatenating shorter words in the list. Right?"

"A few quick questions:

Must use at least 2 words for the concatenation? Yes."

"Let me walk: words=['cat','cats','catsdogcats','dog','dogcatsdog'] -> ['catsdogcats','dogcatsdog'] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for word in words:
    def canBuild(w):
        try every split into a prefix that is
            another word and a remainder that
            is canBuild
return the words that are built from at least two
    words
```

TIME	SPACE
O(n * 2^m)	O(m)

WHY IT IS SLOW

Re-solves the same suffixes. Put the words in a hash set and use a DP over prefixes of each word (dp[i] true if some dp[j] is true and w[j..i] is a word): O(n * m^2).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Sort by length + DP hash map — O(n * L²). Walk-through:"*

Sort words by length. For each word, check if it's formed by concatenation of shorter words.

`dp = set() (initialized with first word or empty – only shorter words used to form longer ones).`

`word_set = set(words) for O(1) lookup.`

For each word (sorted by length):

For split in 1..len(word)-1:

`prefix = word[:split].`

If prefix in word_set and word[split:] in word_set: add word to result; break.

(Need to verify that components are strictly shorter — sort ensures this)

Actually: use dynamic programming: canForm[word] = any prefix in word_set AND canForm[suffix].

WHY sort by length: when checking word, all its components are shorter (already processed).

Time: O(n * L²). Space: O(n * L).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _472_ConcatenatedWords:
    def findAllConcatenatedWordsInADict(self, words):
        words.sort(key=len)
        word_set = set()
        result = []
        for word in words:
            if not word:
                continue
            n = len(word)
            dp = [False] * (n + 1)
            dp[0] = True
            for i in range(1, n + 1):
                for j in range(i):
                    if dp[j] and word[j:i] in word_set:
                        dp[i] = True
```

```
                    break
            if dp[n]:
                result.append(word)
            word_set.add(word)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace the trie navigation (prefix matches) and how it decides the answer."

words=['cat','cats','catsdogcats','dog','dogcatsdog']:

sorted: ['cat','dog','cats','dogcatsdog','catsdogcats']. cat: dp[3]=F(no shorter words yet). -> not concatenated, add to set.

'dog': same. 'cats': dp: j=0,s[0:4]='cats' not in {cat,dog}. j=3,s[3:4]='s' not in set. dp[4]=F. add 'cats'.

'catsdogcats'(11): dp[3]=T('cat'). dp[9]=T('dog'). dp[11]=T('cats'). -> concatenated ok

Processing in length order ensures sub-words are in word_set before longer words

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
$O(n * L^2)$	$O(n*L)$ word set

Follow-up: Word Break (#139) — single word; same DP.

Follow-up: Word Break II (#140) — return all concatenation paths.

HARD | Trie | LeetCode #588 | 6 of 8 | 6/8

Design In-Memory File System

QUESTION

Design a data structure that simulates an in-memory file system.

Implement the FileSystem class:

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Design an in-memory file system with:

`ls(path)->sorted list of files/dirs at path (or just the filename if path is a file),`

`mkdir(path), addContentToFile(path, content)->create/append, readContentFromFile(path). Right?"`

"A few quick questions:

'/' is always implicitly the root dir? Yes. Parent directories for files always exist? Yes per constraints."

"Let me walk: mkdir('/a/b/c'). addContentToFile('/a/b/c/d','hello'). ls('/')->['a']. read->'hello' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
keep every file and directory as a full path in a
    flat map
def ls(path):
    scan all paths for those directly under it
```

TIME	SPACE
O(N * L) per ls	O(N * L)

WHY IT IS SLOW

Every listing scans all N stored paths. A tree of directory nodes (a trie over path segments) makes ls O(path length + entries) and mkdir or addContent O(path length).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) prefix scan for ls.

Trie where each node maps component names to child nodes and stores file content.

ls: if node has content it's a file; else return sorted children keys.

All ops O(L) where L = number of path components.

Space: O(total unique path components).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _588_DesignInMemoryFileSystem:
    def __init__(self):
        self._root = {'_content': None}

    def _navigate(self, path, create=False):
        parts = [p for p in path.split('/') if p]
        node = self._root
        for part in parts:
            if part not in node:
                if not create: return None
                node[part] = {'_content': None}
            node = node[part]
        return node

    def ls(self, path):
        node = self._navigate(path)
        if node['_content'] is not None:
            return [path.split('/')[-1]]
        return sorted(k for k in node if k != '_content')

    def mkdir(self, path):
        self._navigate(path, create=True)

    def addContentToFile(self, path, content):
        node = self._navigate(path, create=True)
        node['_content'] = (node['_content'] or '') + cont
ent

    def readContentFromFile(self, path):
        return self._navigate(path)['_content']
```

PHASE 5 TEST & DEBUG

"Let me trace the trie navigation (prefix matches) and how it decides the answer."

`mkdir('/a/b/c'): root->a->b->c nodes created.`

`addContentToFile('/a/b/c/d','hello'): root->a->b->c->d. d['_content']='hello'.`

`ls('/'): root children (minus '_content'): ['a'] -> sorted=['a'] ok`

`readContentFromFile('/a/b/c/d')->'hello' ok`

`addContentToFile('/a/b/c/d',' world'): d['_content']='hello world'.`

'_content' key distinguishes files (non-None) from directories (None)

Trace: mkdir("/a/b") -> creates nodes a,b. addContentToFile("/a/b/c","hello") -> creates node c with content="hello". ls("/a/b") -> ["c"]. readContentFromFile("/a/b/c") -> "hello" ok.

PHASE 6 COMPLEXITY & FOLLOW-UP

All ops O(L): L = depth of path. Space O(total path components).

Trie shares common prefixes — more memory efficient than storing full path strings.

Follow-up: deleteFile/deleteDir — unlink node from parent, handle subtrees.

PHASE 6 COMPLEXITY & FOLLOW-UP

Follow-up: support wildcard ls (e.g. '/a/*/c') — DFS matching levels.

IN YOUR APP | Expose | Editorial | Community

HARD | Trie | LeetCode #642 | 7 of 8 | 7/8

Design Search Autocomplete System

QUESTION

Design a search autocomplete system for a search engine. Users may input a sentence (at least one word and end with a special character '#').

You are given a string array sentences and an integer array times both of length n where sentences[i] is a previously typed sentence and times[i] is the corresponding number of times the sentence was typed. For each input character except '#', return the top 3 historical hot sentences that have the same prefix as the part of the sentence already typed.

Here are the specific rules:

CONSTRAINTS

- The hot degree for a sentence is defined as the number of times a user typed the exactly same sentence before.
- The returned top 3 hot sentences should be sorted by hot degree (The first is the hottest one). If several sentences have the same hot degree, use ASCII-code order (smaller one appears first).
- If less than 3 hot sentences exist, return as many as you can.
- When the input is a special character, it means the sentence ends, and in this case, you need to return an empty list.

Implement the AutocompleteSystem class:

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given historical sentences with counts and a stream of characters:

On each char (not '#'): return top 3 matching sentences by count desc, lex asc for ties.

On '#': save the typed sentence and reset. Right?"

"A few quick questions:

After '#', the current typed string is added with count 1 (or incremented if exists)?

Yes. Return [] on '#' input? Yes."

"Let me walk: input('i')->['i love you','island','i love leetcode']. input('#')->[] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
keep every sentence in a list with its count
for character in characters:
    scan all sentences for the typed prefix, sort
        the matches by (count descending, text)
        and return the top 3
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(N * L + N log N) per character	O(N * L)

WHY IT IS SLOW

Rescans and re-sorts for every keystroke. A trie where each node keeps its candidate sentences (or the current node is followed one character at a time) answers each keystroke in about O(L).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n*L) prefix scan per character.

Trie where each node caches the top-3 sentences by hot degree.

input(c): navigate trie by one character, read cached top-3 from that node.

input('#'): increment count in counts dict, update trie top-3 for all prefixes.

Time: O(L * k log k) per char where k = candidates at node. Space: O(n*L).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _642_AutocompleteSystem:
    def __init__(self, sentences, times):
        self._counts = {}
        self._prefix = ''
        for s, t in zip(sentences, times):
            self._counts[s] = t

    def _top3(self, prefix):
        matches = [(cnt, s) for s, cnt in self._counts.ite
ms() if s.startswith(prefix)]
        return [s for _, s in sorted(matches, key=lambda x
: (-x[0], x[1]))[:3]]

    def input(self, c):
        if c == '#':
            self._counts[self._prefix] = self._counts.get(
self._prefix, 0) + 1
            self._prefix = ''
            return []
        self._prefix += c
        return self._top3(self._prefix)
```

PHASE 5 TEST & DEBUG

"Let me trace the trie navigation (prefix matches) and how it decides the answer."

sentences=['i love you','island','iroman','i love leetcode'], times=[5,3,2,2]:

input('i'): prefix='i'. matches: 'i love you'(5),'island'(3),'iroman'(2),'i love leetcode'(2).

sorted: (5,'i love you'),(3,'island'),(2,'i love leetcode')[lex:'i love leetcode'<'iroman'].

PHASE 5 TEST & DEBUG

-> ['i love you','island','i love leetcode'] ok

```
input(' '): prefix='i '. matches: 'i love you','i love
  leetcode'. -> [both] ok
input('#'): counts['i ']+=1. prefix=''. -> [] ok
```

Startswith filtering handles prefix matching correctly including spaces

PHASE 6 COMPLEXITY & FOLLOW-UP

```
input(c): O(n*L) scan + O(n log n) sort. For
  production: Trie with cached top-3 per node.
```

Trie: O(L) per char with cached top-3, O(n*L) to update on '#'.

Follow-up: add a delete sentence operation — decrement count, remove if zero.

Follow-up: if sentences are very long, limit Trie depth to max prefix length."

Space: O(n×|?|) — trie stores n characters with branching factor |alphabet|.

IN YOUR APP | Expose | Editorial | Community

HARD | Trie | LeetCode #1032 | 8 of 8 | 8/8

Stream of Characters

QUESTION

Design an algorithm that accepts a stream of characters and checks if a suffix of these characters is a string of a given array of strings words.

For example, if words = ["abc", "xyz"] and the stream added the four characters (one by one) 'a', 'x', 'y', and 'z', your algorithm should detect that the suffix "xyz" of the characters "axyz" matches "xyz" from words.

Implement the StreamChecker class:

CONSTRAINTS

- StreamChecker(String[] words) Initializes the object with the strings array words.
- boolean query(char letter) Accepts a new character from the stream and returns true if any non-empty suffix from the stream forms a word that is in words.
- 1 <= words.length <= 2000
- 1 <= words[i].length <= 200
- words[i] consists of lowercase English letters.
- letter is a lowercase English letter.
- At most $4 * 10^4$ calls will be made to query.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Stream of characters arrives one at a time. query(letter) returns true if any word in dictionary

is a suffix of the stream so far. Right?"

"A few quick questions:

Suffix means the stream ends with that word? Yes."

"Let me walk: words=['ab','ba','oo'], stream: query(a)->F, query(b)->T (stream='ab' ends with 'ab') ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
result.append(whole_stream)
def query(letter):
    append it and check every word against the
        end of the stream
```

TIME	SPACE
O(W * L) per query	O(stream + words)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Compares every word at every query. Build a trie of the reversed words and walk the stream's recent letters backwards through it: O(max word length) per query.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Trie of reversed strings, check suffix matches — O(L) per query. Walk-through:"

Insert all words into trie in REVERSE order.

For each character in the stream:

Maintain a list of trie nodes currently being matched.

For each new character c:

Advance all existing nodes along edge c.

Start a new node from trie root (new potential match starting here).

If any node is a terminal (word end): return True.

WHY reverse: we're checking if any word ends at the current stream position (i.e., is a suffix).

Reversed words in trie: forward traversal of stream chars = backward match of reversed words.

WHY multiple nodes: each stream position starts a potential new suffix match.

Time: O(sum of word lengths) build; O(1) per character with node tracking. Space: O(trie size).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1032_StreamOfCharacters:
    def __init__(self, words):
        self._trie = {}
        for word in words:
            node = self._trie
            for ch in reversed(word):
                node = node.setdefault(ch, {})
            node['#'] = True
        self._stream = []

    def query(self, letter):
        self._stream.append(letter)
        node = self._trie
        for ch in reversed(self._stream):
            if ch not in node:
                return False
            node = node[ch]
            if '#' in node:
                return True
        return False
```

PHASE 5 TEST & DEBUG

"Let me trace the trie navigation (prefix matches) and how it decides the answer."

words=['ab','ba'], stream queries: a->stream=[a]. reversed='a'. trie: 'b'->{'a':{'#':T}}. 'a' not in trie root -> False ok.

b->stream=[a,b]. reversed='b','a': b in trie->go to {'a':{'#':T}}. a in node->go to {'#':T}. '#' found -> True ok.

Reversed Trie matches suffixes; we check stream reversed from current character backward

Edge: stream char not starting any word -> always False until suffix matches. Trie reversed so O(max_word_len) per query vs O(stream_len) naive search ok.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(L) per query where L = max word length	**O(total dictionary chars)**

Aho-Corasick: O(1) per query after O(n*L) build — best for many queries.

Follow-up: return all matching words — collect all '#' nodes during traversal.

Follow-up: if stream can be cleared, reset self._stream.